

60-Day Full-Stack Developer Job-Switch Roadmap

Goal: strengthen your existing production experience and prepare for higher-paying Full-Stack / Backend / Software Engineer roles.

Current package: ■3.6 LPA **Target:** roughly ■10–12 LPA, with higher-paying opportunities as upside.

Your existing strengths: React/Next.js, Node.js, TypeScript/JavaScript, databases, backend APIs, Linux, PM2, Nginx, Docker, AWS/DigitalOcean/Hostinger, Cloudflare/R2, Razorpay, production debugging, 40K+ design-platform users, and election datasets of about 12 crore records.

How to use this roadmap

Use approximately 2–3 hours on weekdays and 4–5 hours on Saturday/Sunday. The plan emphasizes 30% learning, 50% building, and 20% interview preparation. Start applying around Week 3–4 rather than waiting until Day 60.

60-Day Overview

Days	Focus
1–7	TypeScript + Node.js Deep Dive
8–15	SQL + Database Optimization
16–22	Redis + Caching + Queues
23–30	System Design
31–38	Docker + AWS + DevOps
39–46	DSA for Interviews
47–53	React + Next.js + Frontend Architecture
54–60	Resume + Projects + Mock Interviews + Job Switch

PHASE 1 — TypeScript + Node.js Deep Dive — Days 1–7

You already know JavaScript/Node, so focus on depth rather than beginner tutorials.

Day 1 — TypeScript fundamentals

- type, interface, union/intersection, optional properties, enums, tuples, generics, unknown vs any, type narrowing.
- Practice a typed User model and a Promise API function.

Day 2 — Advanced TypeScript

- Generics, keyof, typeof, Partial, Pick, Omit, Record, ReturnType, Parameters.
- Build a reusable ApiResponse type.

Day 3 — Node.js internals

- Event loop, call stack, callback queue, microtasks, Promises, async/await, worker threads, CPU-bound vs I/O-bound work.
- Be able to explain why Node.js can handle many concurrent I/O requests.

Day 4 — Express/Nest architecture

- Controllers, services, repositories, middleware, guards, interceptors, DTOs, validation and error handling.
- Use a clean controllers/services/repositories/dto/utils structure.

Day 5 — Authentication

- JWT, access/refresh tokens, HTTP-only cookies, password hashing, RBAC and authorization.

Day 6 — API design

- REST, HTTP methods/status codes, pagination, filtering, sorting, searching, versioning, rate limiting and idempotency.

Day 7 — Mini project

- Build a production-style User API with registration, login, JWT, refresh token, RBAC, pagination, filtering, validation, error handling and PostgreSQL.

PHASE 2 — SQL + Database Optimization — Days 8–15

This is especially valuable because you already work with very large datasets.

Day 8 — SQL

- JOIN, GROUP BY, HAVING, subqueries, CTEs and window functions. Practice 15–20 queries.

Day 9 — Indexes

- Primary, composite, unique, partial and covering indexes; index selectivity; when indexes help and hurt.

Day 10 — EXPLAIN

- EXPLAIN, EXPLAIN ANALYZE, sequential scan, index scan, bitmap scan, nested loop, hash join and sort. Optimize one real slow query.

Day 11 — Transactions

- ACID, commit/rollback, isolation levels, dirty reads, non-repeatable reads, phantom reads and deadlocks.

Day 12 — Database scaling

- Replication, read replicas, primary/replica architecture, partitioning, sharding and connection pooling.

Day 13 — Pagination

- Offset vs cursor pagination; understand why cursor pagination is often better for large datasets.

Day 14 — PostgreSQL

- JSONB, arrays, CTEs, indexes, transactions, extensions and connection pooling. Understand PgBouncer.

Day 15 — Database project

- Add indexes, cursor pagination, search, filters, transactions and query optimization to your mini project.

PHASE 3 — Redis + Caching + Queues — Days 16–22

Use this phase to turn practical backend work into stronger scalable-system knowledge.

Day 16 — Redis fundamentals

- Strings, hashes, lists, sets, sorted sets, TTL and expiration. Practice SET, GET, DEL, EXPIRE and TTL.

Day 17 — Caching

- Cache-aside, write-through and write-back. Implement request → cache → database → cache flow.

Day 18 — Real application caching

- Cache categories, trending templates, festivals, languages and configuration.

Day 19 — Cache problems

- Cache invalidation, cache stampede, cache penetration, cache avalanche, TTL and distributed locking.

Day 20 — Queues

- Background jobs, producer/consumer, retry and dead-letter queues.

Day 21 — BullMQ

- Queues, workers, jobs, retries, delayed jobs, failed jobs and concurrency. Build a notification queue with FCM.

Day 22 — Mini production architecture

- Combine Node.js + PostgreSQL + Redis + BullMQ in a small working system.

PHASE 4 — System Design — Days 23–30

This is one of the highest-value areas for your next career step because you already have real production systems.

Day 23 — Fundamentals

- Scalability, availability, reliability, latency, throughput, CAP theorem, horizontal vs vertical scaling.

Day 24 — Load balancing

- Reverse proxy, Nginx, load balancing, health checks and sticky sessions.

Day 25 — CDN + object storage

- CDN, CloudFront, S3, Cloudflare/R2, signed URLs and public/private objects.

Day 26 — URL shortener

- Design shortening/redirect APIs; consider database, ID generation, cache, read/write ratio and scaling.

Day 27 — Instagram-like image system

- Client → API → database → object storage → CDN; include upload, resize, compression and metadata.

Day 28 — Notification system

- API → queue → workers → FCM; consider 10K, 100K and 1M users.

Day 29 — Large voter system

- Design search for ~12 crore records using indexes, partitioning/search, cache, read replicas, pagination, rate limiting and connection pooling.

Day 30 — Mock system design

- Design a platform used by 1M users to download social-media designs and videos. Explain every component.

PHASE 5 — Docker + AWS + DevOps — Days 31–38

Build cloud/deployment depth around the infrastructure you already manage.

Day 31 — Docker

- Images, containers, Dockerfiles, volumes, networks and environment variables. Containerize frontend, backend, PostgreSQL and Redis.

Day 32 — Docker Compose

- Create a complete local stack with backend, PostgreSQL and Redis.

Day 33 — AWS EC2

- EC2, security groups, SSH, IAM basics, EBS and Elastic IP. Deploy your backend.

Day 34 — RDS + S3

- RDS, backups, security, S3, bucket policies and presigned URLs. Understand R2 vs S3.

Day 35 — CloudFront/CDN

- Caching, TTL, invalidations and CDN distributions.

Day 36 — Nginx + HTTPS

- Reverse proxy, SSL, domain, HTTP→HTTPS and proxy headers.

Day 37 — CI/CD

- GitHub Actions: test → build → Docker image → deploy → restart.

Day 38 — Monitoring

- Logs, metrics, CPU, memory, disk, latency, error rate and health checks; learn CloudWatch and PM2 monitoring basics.

PHASE 6 — DSA for Interviews — Days 39–46

Spend about 1–1.5 hours/day. Aim for understanding rather than solving hundreds of random problems.

Day 39

- Arrays: Two Sum, maximum subarray, remove duplicates.

Day 40

- Strings: anagrams, longest substring, character frequency.

Day 41

- HashMap/Set: frequency, duplicates and lookup optimization.

Day 42

- Stack/Queue: valid parentheses, min stack and queue problems.

Day 43

- Binary search: basic, rotated array and search-answer patterns.

Day 44

- Linked list: reverse, cycle detection and merge lists.

Day 45

- Trees: DFS, BFS, binary-tree traversal and height/depth.

Day 46 — Revision

- Target roughly 40–50 good problems. For each, know brute force, optimized approach, time complexity and space complexity.

PHASE 7 — React + Next.js + Frontend Architecture — Days 47–53

Polish your existing frontend skills rather than learning another framework.

Day 47 — React internals

- Rendering, reconciliation, Virtual DOM, state, props and context.

Day 48 — Performance

- useMemo, useCallback, React.memo, lazy loading, code splitting and bundle optimization; understand when not to memoize.

Day 49 — Next.js

- SSR, CSR, SSG, ISR, Server Components, Server Actions and routing.

Day 50 — Frontend architecture

- components, features, hooks, services, utils and types; structure large applications cleanly.

Day 51 — State management

- Context, Redux Toolkit, server state and TanStack Query; distinguish client state from server state.

Day 52 — Frontend security

- XSS, CSRF, CORS, authentication, authorization and secure cookies.

Day 53 — Dashboard project

- Build a polished dashboard with login, role-based UI, pagination, search, filters, API integration, loading and error states.

PHASE 8 — Interview + Job Switch — Days 54–60

Convert your technical work into a clear, evidence-based interview story.

Day 54 — Resume

- Position yourself as a Full-Stack Developer who handles frontend, backend, database, deployment and production infrastructure. Mention 40K+ users and ~12 crore records only where you can substantiate them.

Day 55 — Design project explanation

- Prepare a 5-minute story: problem → architecture → technology → your responsibility → scale → problems → solutions → result.

Day 56 — Election project explanation

- Be ready to explain database, indexing, query strategy, pagination, search, API, deployment, backup and performance challenges for the ~12 crore-record system.

Day 57 — Backend interview

- Practice Node.js, Express/NestJS, REST, authentication, Redis, SQL, transactions and API design.

Day 58 — System design interview

- Do two mocks: social-media design platform and 12-crore-record voter search system.

Day 59 — HR/behavioral

- Prepare: tell me about yourself, why leaving, current salary, expected salary, production problem, independently handled project and biggest technical challenge.

Day 60 — Job application launch

- Target Full-Stack Developer, Software Engineer, Backend Developer, Node.js Developer, React + Node Developer and TypeScript Developer roles. Start applications around Week 3–4; do not wait until Day 60.

Recommended Daily Schedule

Monday–Friday: 45 min DSA + 1.5 hr main topic + 20–30 min revision/interview questions. Total ~2.5–3 hours/day.

Saturday: 2 hr main topic + 2 hr project + 1 hr DSA.

Sunday: 2 hr system design + 1.5 hr project + 1 hr revision + 1 hr interview practice.

Important: do not spend these 60 days only watching tutorials. Use 30% learning, 50% building, 20% interview preparation.

Your final target profile

Area	Target
Frontend	React, Next.js, TypeScript, Redux Toolkit, performance
Backend	Node.js, NestJS/Express, REST, auth, queues
Database	PostgreSQL/MySQL, indexing, transactions, optimization
Caching	Redis, TTL, invalidation, queues
System Design	Scaling, load balancing, CDN, storage, caching, databases
DevOps	Docker, Nginx, PM2, CI/CD, monitoring
Cloud	AWS EC2/RDS/S3/CloudFront + Cloudflare/R2 concepts
Interview	DSA + system design + project storytelling

Final strategy: Your advantage is not simply knowing many technologies. It is that you already have production experience. Use these 60 days to add deeper engineering knowledge and become much better at explaining why you chose each technology, how you handled scale, and how you would improve the system.

Start interviewing before you feel 100% ready. Interview feedback should become part of the learning loop.